Phase 1 Background

Assume student information and digital services are handled in separate tools. Three problems are inconsistent registration checks, repeated manual score calculation, and poorly targeted notifications. The examination solution centralizes registration decisions and separates evaluation and integration logic behind interfaces.

Objectives

Primary objective: Develop an object-oriented campus subsystem that validates registrations, computes scores using interchangeable policies, sends appropriate notifications, and accesses an external library through a stable interface.

1. Create supported account roles through one creation interface.
2. Enforce registration capacity, duplication, and credit constraints.
3. Calculate and publish scores using a selected evaluation policy.
4. Integrate library availability and distinguish public notices from personal results.

Actors

Actor	Interaction
Student	Registers for courses, receives notices and personal results, and checks book availability.
Faculty Member	Supplies assessment marks and chooses the appropriate evaluation policy.
Administrator	Creates account roles and maintains campus configuration and course information.
Librarian	Maintains the external library catalogue used through an adapter.

System boundary: the platform contains domain records, controllers, portals, and adapters. External people, third-party systems, and physical devices lie outside it. A portal or adapter is an internal boundary component, not the external actor itself.

Security Officer and IoT System are broader-platform actors for access monitoring and attendance. The model does not implement their hardware interactions.

Functional requirements

FR1: Create Student, Faculty, and Administrator accounts through UserAccount and reject unsupported roles.
FR2: Register a student only when the course is not already selected, a seat is available, and the resulting load is at most six credits.
FR3: Validate assessment marks, compute scores with the selected policy, and notify the matching student.
FR4: Broadcast notices, remove subscribers, and translate external library counts into availability.

Non functional requirements

Maintainability: evaluation algorithms and library-provider integration must be replaceable through interfaces.
Reliability: rejected registration must not change enrollment or credit totals; invalid marks must not produce a result notification.

Phase 2 Object Identification Solution

Entity objects

Entity	Attributes	Implementation scope
Student	id, role	Implemented account and domain representation; role returned by role().
Course	code, credits, capacity, students	Implemented course information and enrolled student IDs.
Marks	exam, coursework, project	Implemented assessment value object.
LibraryBook	isbn, title, availableCopies	Extension; provider exposes copy counts in the example.
Notice	noticeId, message, recipientId	Extension; code passes recipient and message directly.

Control objects

Class	Responsibility
CourseRegistrationManager	Validate and record course registration and credit totals.
EvaluationService	Delegate calculation to the active policy and publish a result.
NoticePublisher	Coordinate registration, removal, and delivery of notification events.

Boundary objects

Class	Interaction
StudentPortal	Display public notices and results matching the student ID.
FacultyPortal	Display public campus notices in this teaching example.
LibraryAdapter	Bridge the incompatible external library query format.

Phase 3 CRC Card Solution

CRC Cards – Experiment 3 Solution (SUCMS Lab Exam)

CourseRegistrationManager	
Responsibilities	**Collaborators**
• Check duplicate registration, capacity, and credit limit; • update enrollment and credit totals.	Student, Course, UniversityConfig

Student	
Responsibilities	**Collaborators**
• Retain account identity; • report the student role; • identify the owner of registrations and results.	UserAccount, AccountFactory, CourseRegistrationManager, EvaluationService

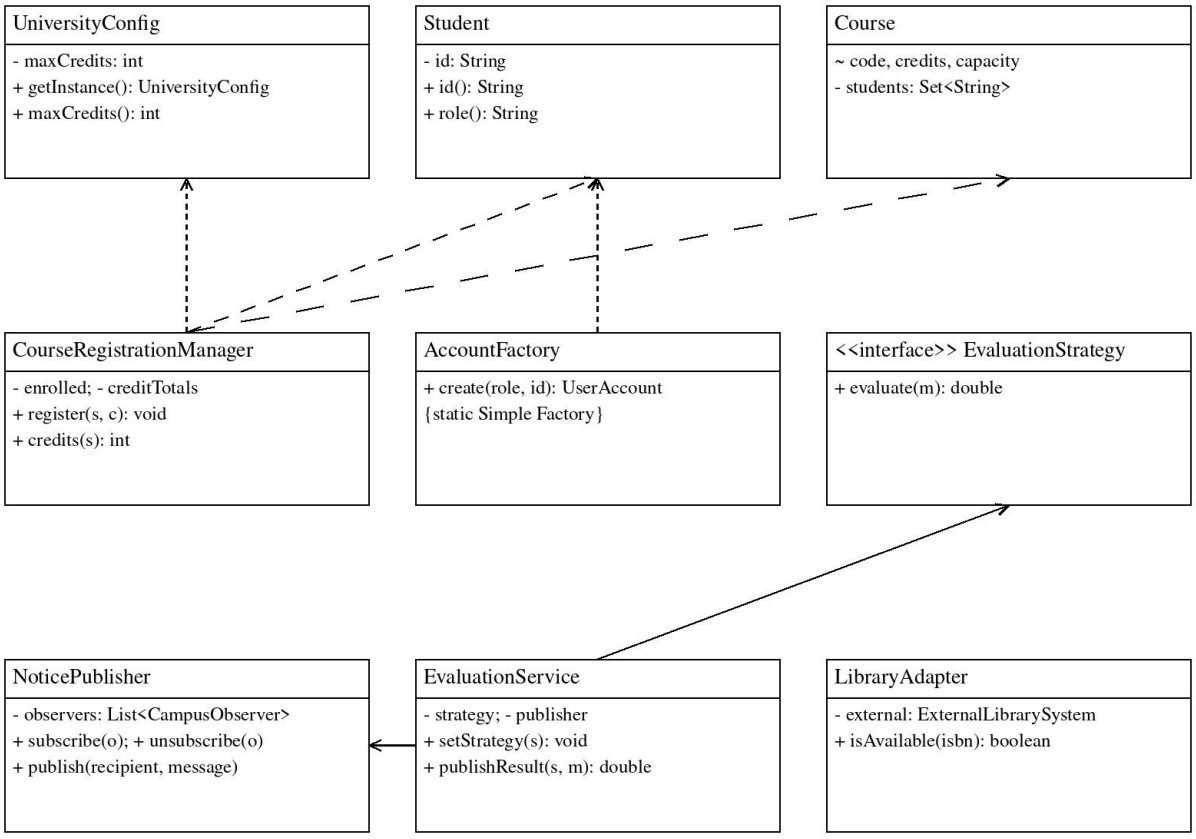
Course	
Responsibilities	**Collaborators**
• Retain code, credits, and capacity; • retain enrolled student IDs.	CourseRegistrationManager, Student

EvaluationService	
Responsibilities	**Collaborators**
• Replace evaluation policy; • calculate scores; • publish the score for its student.	EvaluationStrategy, Marks, NoticePublisher

NoticePublisher	
Responsibilities	**Collaborators**
• Maintain subscribers; • remove subscribers; • deliver recipient-tagged messages.	CampusObserver, StudentPortal, FacultyPortal, EvaluationService

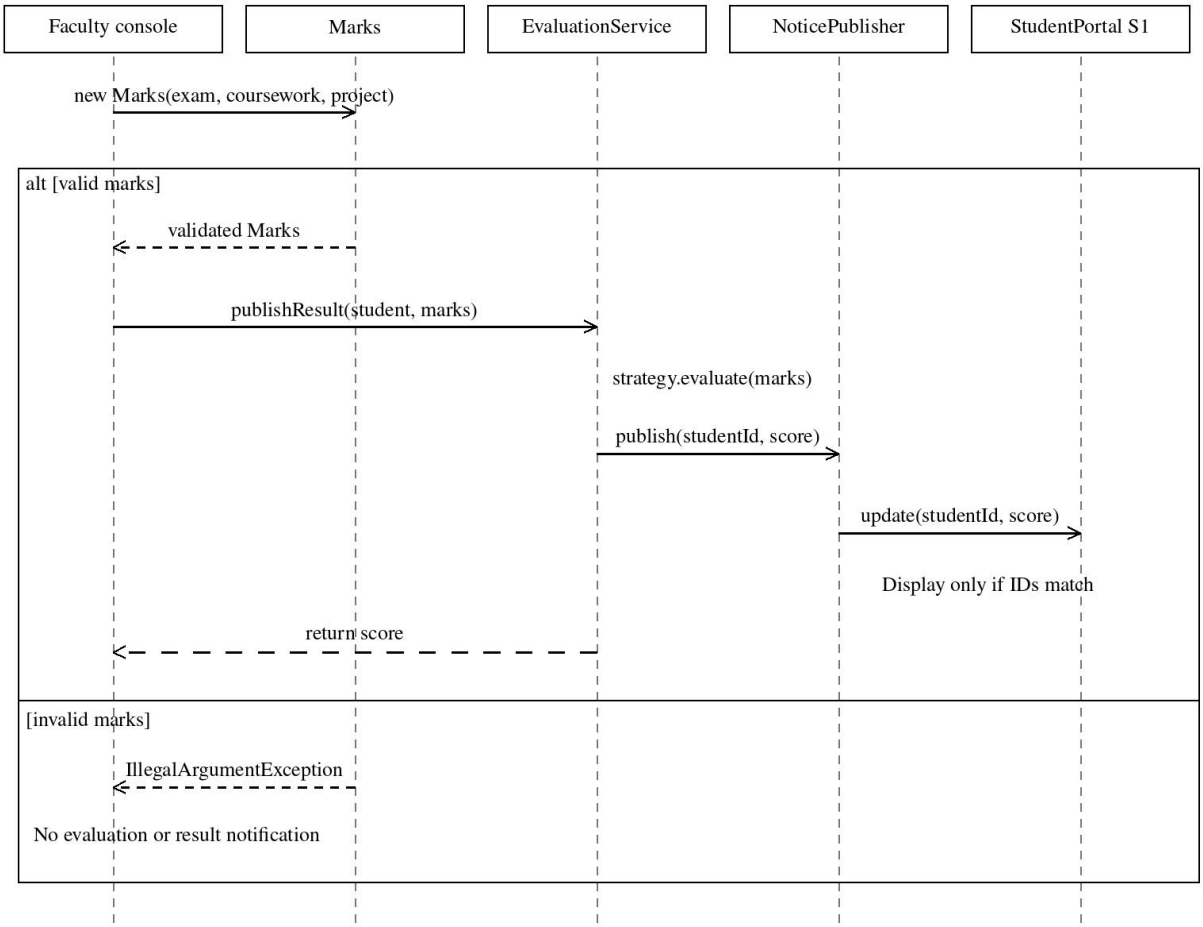
LibraryAdapter	
Responsibilities	**Collaborators**
• Query the provider; • parse and validate copy counts; • return availability or an error.	LibraryService, ExternalLibrarySystem

Phase 4 UML Class Diagram Solution



Additional interface realizations and associations are specified in the table below.

Phase 4 UML Sequence Diagram Solution



Phase 5 Java Solution Listing 1

```
import java.util.*;
public class SUCMSExam {
  static void requireText(String s) {
    if (s == null || s.trim().isEmpty())
      throw new IllegalArgumentException("Blank value");
  }
  static final class UniversityConfig {
    private static final UniversityConfig INSTANCE = new UniversityConfig();
    private final int maxCredits = 6;
    private UniversityConfig() { }
    public static UniversityConfig getInstance() { return INSTANCE; }
    public int maxCredits() { return maxCredits; }
  }
  interface UserAccount { String id(); String role(); }
  static final class Student implements UserAccount {
    private final String id;
    Student(String id) { requireText(id); this.id = id; }
    public String id() { return id; }
    public String role() { return "STUDENT"; }
  }
  static final class Faculty implements UserAccount {
    private final String id;
    Faculty(String id) { requireText(id); this.id = id; }
    public String id() { return id; }
    public String role() { return "FACULTY"; }
  }
  static final class Administrator implements UserAccount {
    private final String id;
    Administrator(String id) { requireText(id); this.id = id; }
    public String id() { return id; }
    public String role() { return "ADMIN"; }
  }
  static final class AccountFactory {
    public static UserAccount create(String role, String id) {
      if ("STUDENT".equals(role)) return new Student(id);
      if ("FACULTY".equals(role)) return new Faculty(id);
      if ("ADMIN".equals(role)) return new Administrator(id);
```

```
    throw new IllegalArgumentException("Unsupported role");
  }
}
static final class Course {
  final String code;
  final int credits, capacity;
  private final Set<String> students = new HashSet<>();
  Course(String code, int credits, int capacity) {
    requireText(code);
    if (credits <= 0 || capacity <= 0) throw new IllegalArgumentException();
    this.code = code; this.credits = credits; this.capacity = capacity;
  }
}
static final class CourseRegistrationManager {
  private final Map<String, Set<String>> enrolled = new HashMap<>();
  private final Map<String, Integer> creditTotals = new HashMap<>();
  public void register(Student s, Course c) {
    Set<String> codes = enrolled.getOrDefault(s.id(), Collections.emptySet());
    int current = creditTotals.getOrDefault(s.id(), 0);
    if (codes.contains(c.code)) throw new IllegalStateException("Duplicate course");
    if (c.students.size() >= c.capacity) throw new IllegalStateException("Full course");
    if (current + c.credits > UniversityConfig.getInstance().maxCredits())
      throw new IllegalStateException("Credit limit");
    enrolled.computeIfAbsent(s.id(), key -> new HashSet<>()).add(c.code);
    c.students.add(s.id()); creditTotals.put(s.id(), current + c.credits);
  }
  public int credits(Student s) { return creditTotals.getOrDefault(s.id(), 0); }
}
interface CampusObserver { void update(String recipient, String message); }
static final class StudentPortal implements CampusObserver {
  private final String studentId;
  int received;
  StudentPortal(String id) { studentId = id; }
  public void update(String recipient, String message) {
    if (recipient != null && !recipient.equals(studentId)) return;
    received++; System.out.println("Student " + studentId + ": " + message);
  }
}
```

```
static final class FacultyPortal implements CampusObserver {
  int received;
  public void update(String recipient, String message) {
    if (recipient != null) return; // Only public notices in this example.
    received++; System.out.println("Faculty: " + message);
  }
}
static final class NoticePublisher {
  private final List<CampusObserver> observers = new ArrayList<>();
  public void subscribe(CampusObserver o) {
    if (!observers.contains(o)) observers.add(o);
  }
  public void unsubscribe(CampusObserver o) { observers.remove(o); }
  public void publish(String recipient, String message) {
    for (CampusObserver o : new ArrayList<>(observers)) o.update(recipient, message);
  }
}
static final class Marks {
  final double exam, coursework, project;
  Marks(double exam, double coursework, double project) {
    for (double x : new double[] {exam, coursework, project})
      if (!Double.isFinite(x) || x < 0 || x > 100)
        throw new IllegalArgumentException("Marks must be 0 to 100");
    this.exam = exam; this.coursework = coursework; this.project = project;
  }
}
interface EvaluationStrategy { double evaluate(Marks m); }
static final class TheoryEvaluation implements EvaluationStrategy {
  public double evaluate(Marks m) { return .6 * m.exam + .4 * m.coursework; }
}
static final class LabEvaluation implements EvaluationStrategy {
  public double evaluate(Marks m) {
    return .3 * m.exam + .2 * m.coursework + .5 * m.project;
  }
}
static final class EvaluationService {
  private EvaluationStrategy strategy;
  private final NoticePublisher publisher;
```

```
  EvaluationService(EvaluationStrategy s, NoticePublisher p) {
    strategy = s; publisher = p;
  }
  public void setStrategy(EvaluationStrategy s) { strategy = s; }
  public double publishResult(Student student, Marks marks) {
    double score = strategy.evaluate(marks);
    publisher.publish(student.id(), "Score=" + score); return score;
  }
}
interface LibraryService { boolean isAvailable(String isbn); }
static final class ExternalLibrarySystem {
  public String copiesAsText(String isbn) {
    if ("BOOK1".equals(isbn)) return "2";
    if ("BOOK2".equals(isbn)) return "0";
    return "UNKNOWN";
  }
}
static final class LibraryAdapter implements LibraryService {
  private final ExternalLibrarySystem external;
  LibraryAdapter(ExternalLibrarySystem external) { this.external = external; }
  public boolean isAvailable(String isbn) {
    requireText(isbn);
    String text = external.copiesAsText(isbn);
    if ("UNKNOWN".equals(text)) throw new IllegalArgumentException("Unknown book");
    int copies = Integer.parseInt(text);
    if (copies < 0) throw new IllegalArgumentException("Invalid copies");
    return copies > 0;
  }
}
static void expect(boolean ok, String label) {
  if (!ok) throw new AssertionError(label);
  System.out.println("PASS: " + label);
}
static void rejects(Runnable action, String label) {
  try { action.run(); }
  catch (IllegalArgumentException | IllegalStateException e) {
    System.out.println("PASS: " + label); return;
  }
```

```
  throw new AssertionError(label);
}
public static void main(String[] args) {
  expect(UniversityConfig.getInstance() == UniversityConfig.getInstance(), "singleton");
  Student student = (Student) AccountFactory.create("STUDENT", "S1");
  Student second = (Student) AccountFactory.create("STUDENT", "S2");
  expect(AccountFactory.create("FACULTY", "F1") instanceof Faculty, "faculty factory");
  expect(AccountFactory.create("ADMIN", "A1") instanceof Administrator, "admin factory");
  rejects(() -> AccountFactory.create("UNKNOWN", "X"), "unsupported account");
  rejects(() -> AccountFactory.create("STUDENT", ""), "blank account ID");
  CourseRegistrationManager manager = new CourseRegistrationManager();
  Course c1 = new Course("C1", 3, 1), c2 = new Course("C2", 3, 5);
  Course c3 = new Course("C3", 3, 5);
  manager.register(student, c1);
  rejects(() -> manager.register(student, c1), "duplicate registration");
  rejects(() -> manager.register(second, c1), "capacity limit");
  manager.register(student, c2);
  expect(manager.credits(student) == 6, "six credit boundary accepted");
  rejects(() -> manager.register(student, c3), "credit limit exceeded");
  expect(manager.credits(student) == 6 && manager.credits(second) == 0,
      "failed registration leaves credits unchanged");
  NoticePublisher publisher = new NoticePublisher();
  StudentPortal p1 = new StudentPortal("S1"), p2 = new StudentPortal("S2");
  FacultyPortal faculty = new FacultyPortal();
  publisher.subscribe(p1); publisher.subscribe(p2); publisher.subscribe(faculty);
  publisher.publish(null, "Library hours updated");
  expect(p1.received == 1 && p2.received == 1 && faculty.received == 1,
      "broadcast notice");
  EvaluationService evaluation = new EvaluationService(new TheoryEvaluation(), publisher);
  Marks m = new Marks(80, 70, 90);
  expect(Math.abs(evaluation.publishResult(student, m) - 76) < 1e-9, "theory score 76");
  expect(p1.received == 2 && p2.received == 1 && faculty.received == 1,
      "result recipient filtering");
  evaluation.setStrategy(new LabEvaluation());
  expect(Math.abs(evaluation.publishResult(student, m) - 83) < 1e-9, "lab score 83");
  publisher.unsubscribe(p2); publisher.publish(null, "Campus notice");
  expect(p2.received == 1 && p1.received == 4 && faculty.received == 2,
      "observer removal");
```

```
    rejects(() -> new Marks(-1, 70, 90), "negative marks rejected");
    rejects(() -> new Marks(101, 70, 90), "marks above 100 rejected");
    rejects(() -> new Marks(Double.NaN, 70, 90), "NaN marks rejected");
    LibraryService library = new LibraryAdapter(new ExternalLibrarySystem());
    expect(library.isAvailable("BOOK1"), "available book");
    expect(!library.isAvailable("BOOK2"), "unavailable book");
    rejects(() -> library.isAvailable("MISSING"), "unknown book");
    System.out.println("All checks completed.");
  }
}
```